

Como ingeniero, para documentar el **Proyecto Socio Tecnológico** (PST) o la defensa de tu sistema, es fundamental explicar que no usaste el sistema de autenticación "cerrado" de Laravel (como Breeze o Jetstream), sino una implementación **personalizada** para tener control total sobre la lógica de seguridad.

Aquí tienes un texto técnico y profesional sobre cómo configuraste el backend para la gestión de contingencias:

Implementación del Módulo de Recuperación de Contraseña (Backend)

Para el sistema de Gestión de Contingencias Satelitales, se desarrolló un flujo de recuperación de credenciales basado en **tokens temporales de un solo uso**, priorizando la seguridad y la integridad de los datos de la plataforma.

1. Arquitectura y Herramientas Utilizadas

El backend, desarrollado en **Laravel 10/11**, utiliza los siguientes componentes para este proceso:

- **Mailables de Laravel:** Se configuró la clase Mailable para estructurar correos electrónicos profesionales con formato HTML, permitiendo el envío automático de códigos de seguridad.
- **Protocolo SMTP:** Integración con servidores de correo para garantizar la entrega de notificaciones en tiempo real.
- **Bases de Datos (PostgreSQL):** Se utiliza una migración específica para gestionar códigos temporales, vinculándolos de forma única al correo electrónico del usuario solicitante.
- **Eloquent ORM:** Para la consulta, validación y actualización segura de las contraseñas mediante el uso de hashes (**Bcrypt**).

2. Funcionamiento del Flujo Técnico

El proceso se divide en dos fases críticas de validación:

Fase A: Generación y Envío del Código

Cuando el usuario solicita la recuperación desde el frontend (ReactJS), el servidor ejecuta las siguientes acciones:

1. **Validación de Identidad:** Verifica que el correo electrónico exista en la tabla de usuarios.

2. **Generación de Clave Temporal:** Se crea un código aleatorio numérico (OTP - One Time Password) de alta entropía.
3. **Persistencia:** El código se almacena en la base de datos asociado al usuario, incluyendo una marca de tiempo para controlar su expiración.
4. **Despacho de Correo:** Se dispara un evento de envío que entrega al usuario su clave única de acceso temporal.

Fase B: Restablecimiento Seguro

Una vez que el usuario recibe el código, el sistema procesa la petición de cambio:

1. **Verificación de Integridad:** Se comparan tres factores: el correo, el código temporal ingresado y la vigencia del mismo.
2. **Hash de Seguridad:** La nueva contraseña nunca se guarda en texto plano; Laravel aplica automáticamente el algoritmo **Bcrypt** antes de almacenarla en PostgreSQL.
3. **Limpieza de Datos:** Tras el éxito del cambio, el código temporal es invalidado o eliminado para prevenir ataques de reutilización.

3. Consideraciones de Seguridad Aplicadas

- **Protección contra Fuerza Bruta:** El sistema limita el número de intentos fallidos al ingresar el código.
- **Validación de Complejidad:** Se aplican reglas estrictas (min:8, caracteres alfanuméricos) para asegurar que la nueva contraseña sea robusta.
- **Separación de Responsabilidades:** La lógica de recuperación reside en un controlador dedicado (AuthController o RecoveryController), manteniendo el código limpio y mantenible bajo el patrón MVC.

Nota para la documentación: Este diseño garantiza que, ante una contingencia, el personal de la plataforma satelital pueda recuperar su acceso de manera autónoma sin comprometer la seguridad global del sistema.

Este texto te sirve perfectamente para el manual técnico o para la sección de "Desarrollo" en tu informe de pasantías o proyecto. ¡Descansa, que mañana terminamos de pulir lo que falta!